2026.06.24 개발 일지

1) 작업 내용 (What I Did)

오늘 수행한 구체적인 작업 내용을 명확하게 기록합니다.

구분	내용
작업 내용 (What I Did)	사용자 정의 Peripheral의 AXI Register Block과 프로토콜 제어 로직을 구현하였다. 주요 수행 내용은 다음과 같다. • UART AXI Register Block의 TX_DATA, RX_DATA, STATUS, CONTROL 접근 동작을 구현하였다. • Timer의 Load, Counter, Enable, Done 상태 레지스터를 구성하였다. • I2C LCD용 Data/Command, Control, Status 레지스터를 정의하였다. • I2C Control FSM과 Master 송수신 제어 신호를 연결하였다. • SPI AXI Register Block의 제어 및 상태 레지스터를 구현하였다. • SPI Master의 Clock Divider, Shift Register, Chip Select, Bit Counter 로직을 작성하였다. UART, Timer, I2C LCD, SPI의 레지스터 접근 경로와 핵심 제어 로직을 마련하여 AXI 통합 검증 단계로 진행할 수 있게 되었다.

2) 회고 및 개선 사항

구분	내용
Keep (잘한 점, 유지할 점)	**Keep** • Peripheral마다 동일한 AXI Register Block 구조를 적용하여 코드 흐름을 일관되게 유지하였다. • 제어 레지스터와 상태 레지스터를 분리하여 소프트웨어 접근 의미를 명확히 하였다. • SPI 데이터 경로와 제어 FSM을 구분해 디버깅 가능성을 높였다. **Problem** • 여러 상태 신호를 AXI Read Data로 결합하는 과정에서 우선순위가 복잡해졌다. • I2C와 SPI는 프로토콜 타이밍 요구가 달라 공통 구조만으로 처리하기 어려웠다.
Try (새롭게 시도할 점, 개선할 점)	레지스터 Decode를 공통 패턴으로 정리하고 Peripheral별 동작만 별도 로직으로 분리할 예정이다. 상태 비트 갱신 조건을 표로 정리하여 테스트벤치 Check Point로 활용할 예정이다.

3) 이슈 및 해결 과정

발생한 Trouble, 장애 상황과 이를 해결하기 위한 과정 및 결과, 혹은 미해결된 경우 현재 상태를 기록합니다.

구분	내용
Trouble (이슈/장애 및 해결 과정)	**Trouble** SPI CONTROL 레지스터의 START 비트와 내부 FSM 시작 조건을 직접 연결할 경우 START가 여러 Cycle 유지될 수 있었다. **Action**

구분	내용
	AXI Write 완료 이벤트를 기준으로 One-Cycle Start Pulse를 생성하고 FSM에는 해당 Pulse만 전달하도록 구조를 변경하였다. **Result** 단일 Register Write마다 SPI 전송이 한 번만 시작되도록 제어 경로를 정리하였다.

3. 토의 필요 사항

팀 미팅이나 기술 토의 시간에 논의가 필요한 사항을 명시합니다. 논의 필요 시 관련 미팅 링크를 추가할 수 있습니다.

- 논의 사항 1: Timer DONE 비트의 자동 Clear 시점
- 논의 사항 2: I2C LCD Command/Data 구분 비트의 레지스터 위치

4. 첨부 자료

개발 중 발생한 산출물이나 참고 자료를 첨부합니다.

확인된 자료 중 해당 날짜의 작업과 직접 연결되는 산출물을 첨부하였다.

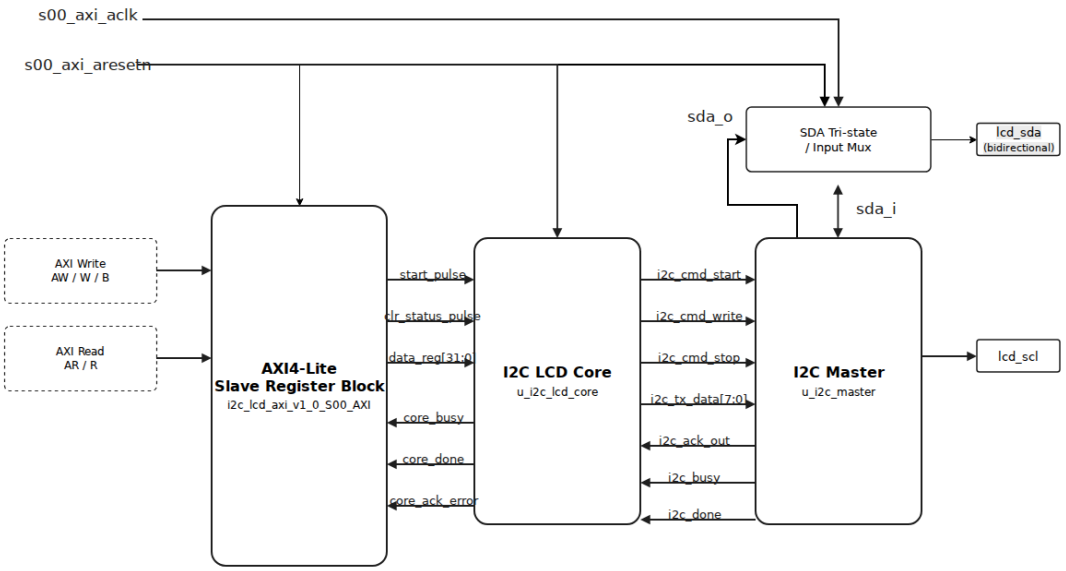


그림 1. AXI4-Lite Register Block과 I2C LCD 제어 구조

출처: VerilogHDL.drawio 「Stopwatch」 페이지 내 I2C LCD 설계 조각 (pageId: EKWhh1RQuC7uWDcN-W7v)

자료 확인 필요: 해당 날짜의 RTL 변경 내역과 Compile·Lint 원본 로그는 제공 자료에서 확인되지 않음.